

16: Pipelining I

ENGR 315: Hardware/Software CoDesign

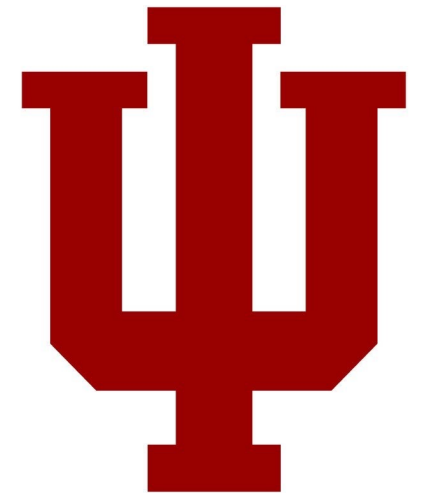
Andrew Lukefahr

Indiana University

Some material taken from:

https://github.com/trekhleb/homemade-machine-learning/tree/master/homemade/neural_network

<http://cs231n.github.io/neural-networks-1/>



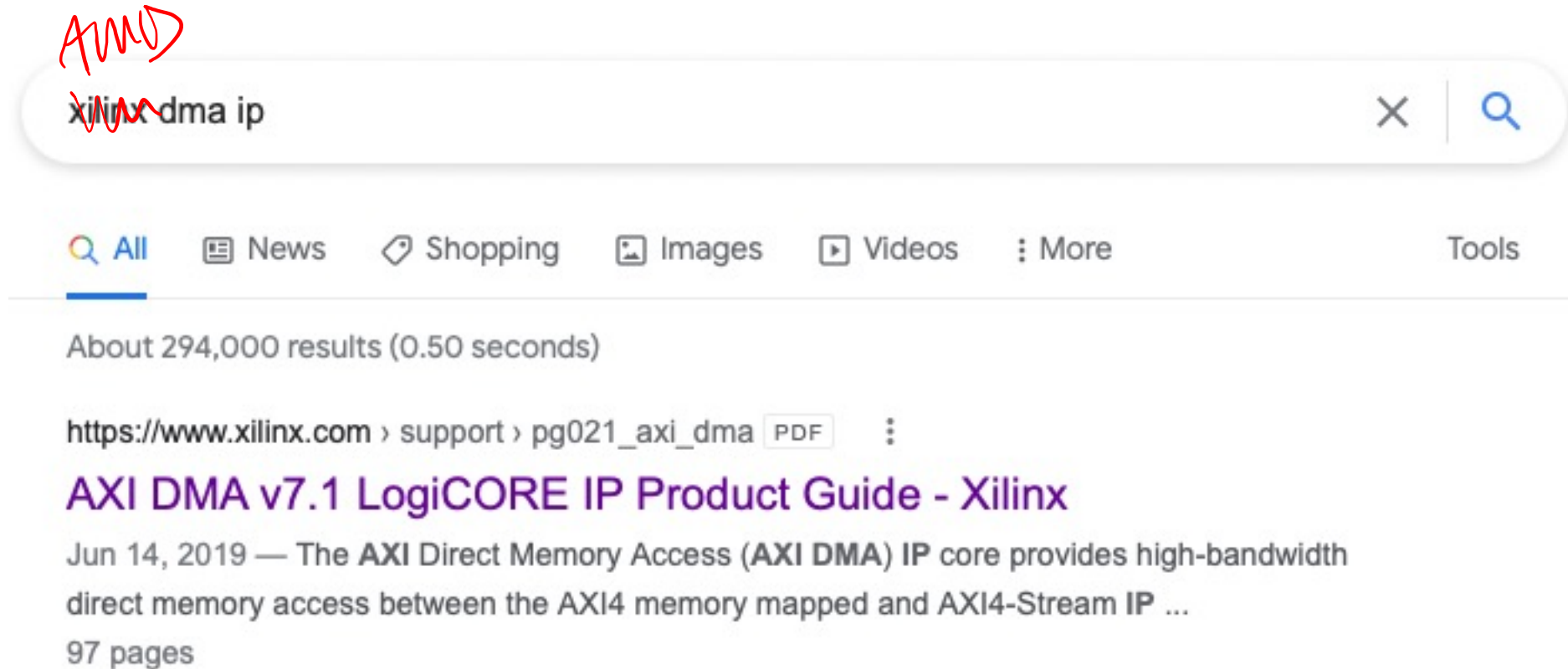
Announcements

- P7 *— last pop count*
 - AG due Nov 5th
- P8:
 - AG out *— Nov 19th*

Exam Planning

11/05	Wednesday	10	Parallelism II / Review	–
11/12	Wednesday	–	Exam	P8 Dot (V)

P7 – DMA from C



P7 – DMA from C

Programming Sequence

Direct Register Mode (Simple DMA)

P7 – DMA from C

1. Start the MM2S channel running by setting the run/stop bit to 1 (MM2S_DMACR.RS = 1). The halted bit (DMASR.Halted) should deassert indicating the MM2S channel is running.

2. Skip

3. Write a valid source address to the MM2S_SA register.
4. Write the number of bytes to transfer in the MM2S_LENGTH register.
The MM2S_LENGTH register must be written last.
5. Wait until MM2S_DMASR.Idle==1 for completion

P8+ Accelerate Machine Learning

- Goal: Accelerate reference neural network
- Harder, more open-ended projects

What Number is this?

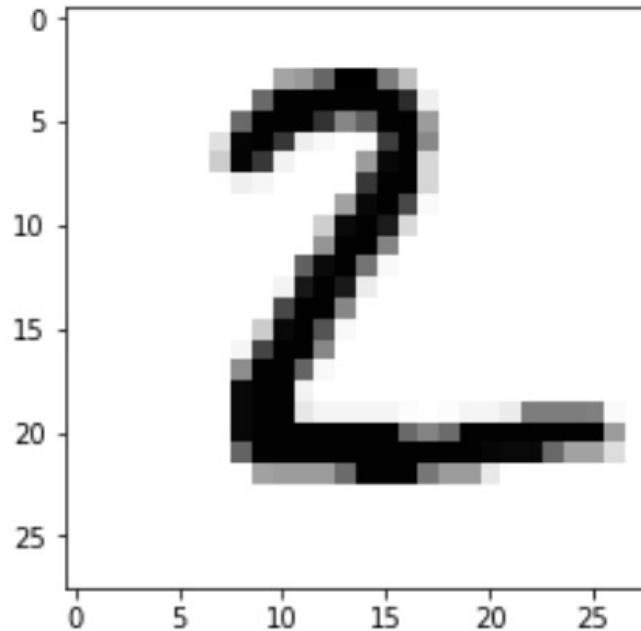
```
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 74 7d ab ff ff 96 5d 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 a9 fd fd fd fd fd fd da 1e 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 a9 fd fd fd d5 8e b0 fd fd 7a 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 34 fa fd d2 20 0c 00 06 ce fd 8c 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 4d fb d2 19 00 00 00 7a f8 fd 41 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 1f 12 00 00 00 00 d1 fd fd 41 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 75 f7 fd c6 0a 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 4c f7 fd e7 3f 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 80 fd fd 90 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 b0 f6 fd 9f 0c 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 19 ea fd e9 23 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 c6 fd fd 8d 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 4e f8 fd bd 0c 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 13 c8 fd fd 8d 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 86 fd fd ad 0c 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 f8 fd fd 19 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 f8 fd fd 2b 14 14 14 14 05 00 05 14 14 25 96 96 96 93 0a 00
00 00 00 00 00 00 00 f8 fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
00 00 00 00 00 00 00 ae fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
00 00 00 00 00 00 00 76 7b 7b 7b a6 fd fd fd 9b 7b 7b 29 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Raw data, in hex

Simple Neural Network

=====
Index: 0

Image:



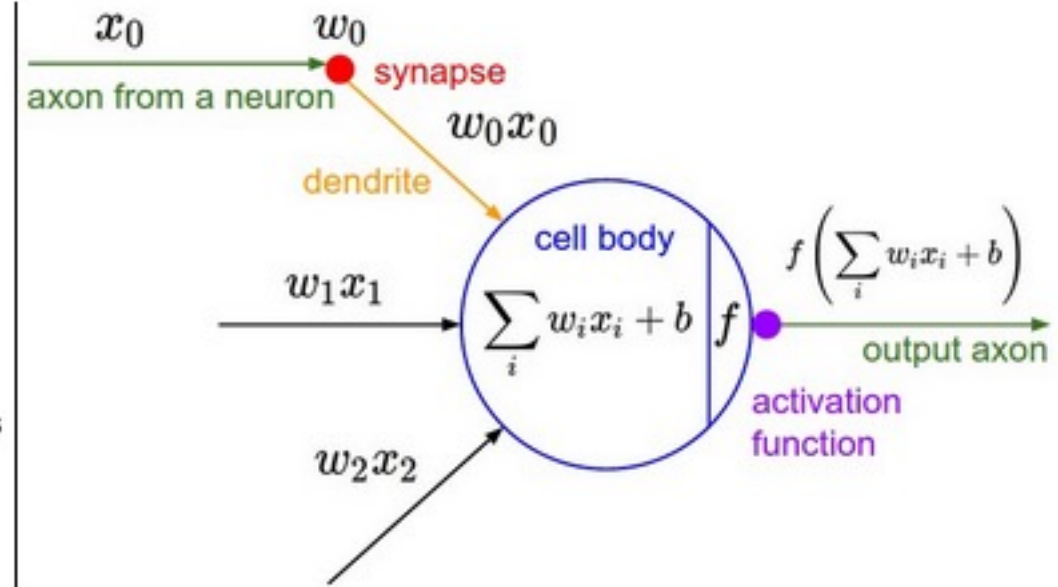
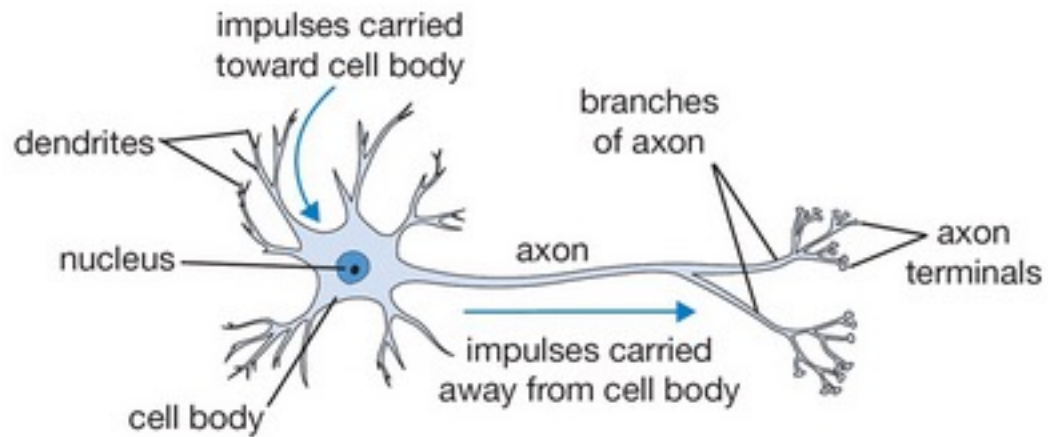
→ ML Classification Result: 2 ←

Real Value: 2

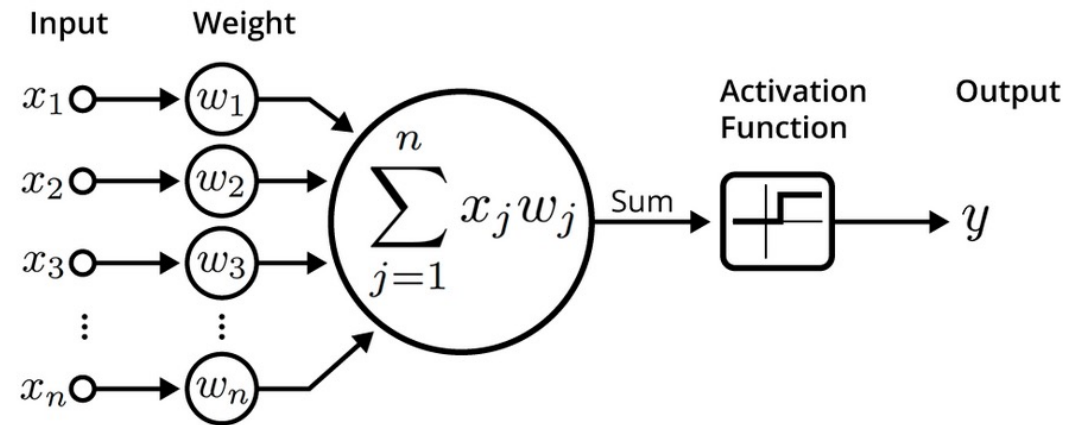
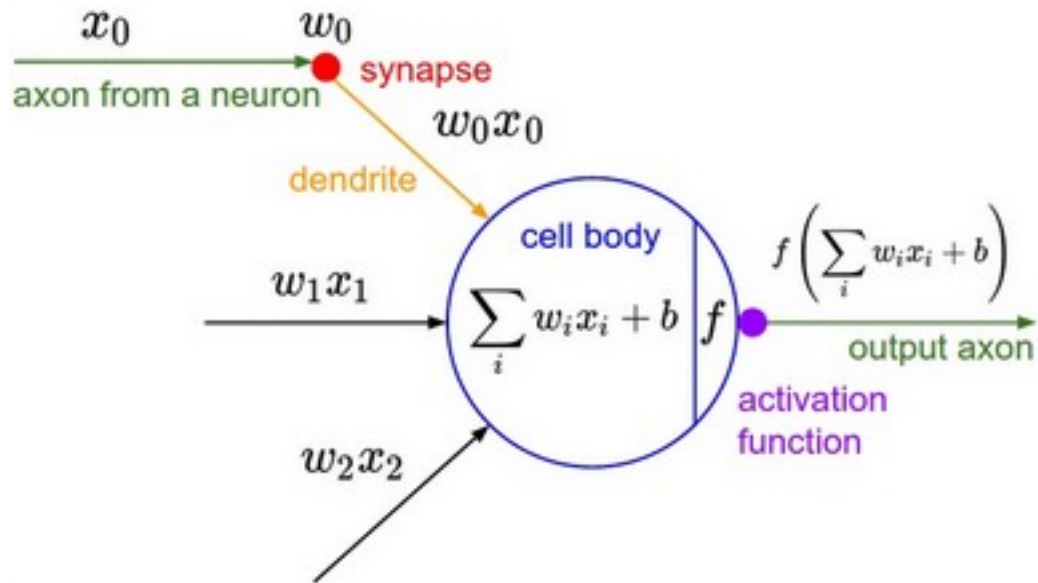
Correct Result: True

- Takes in image of number
- Returns integer value
- How? artificial neural network

The Math Neuron

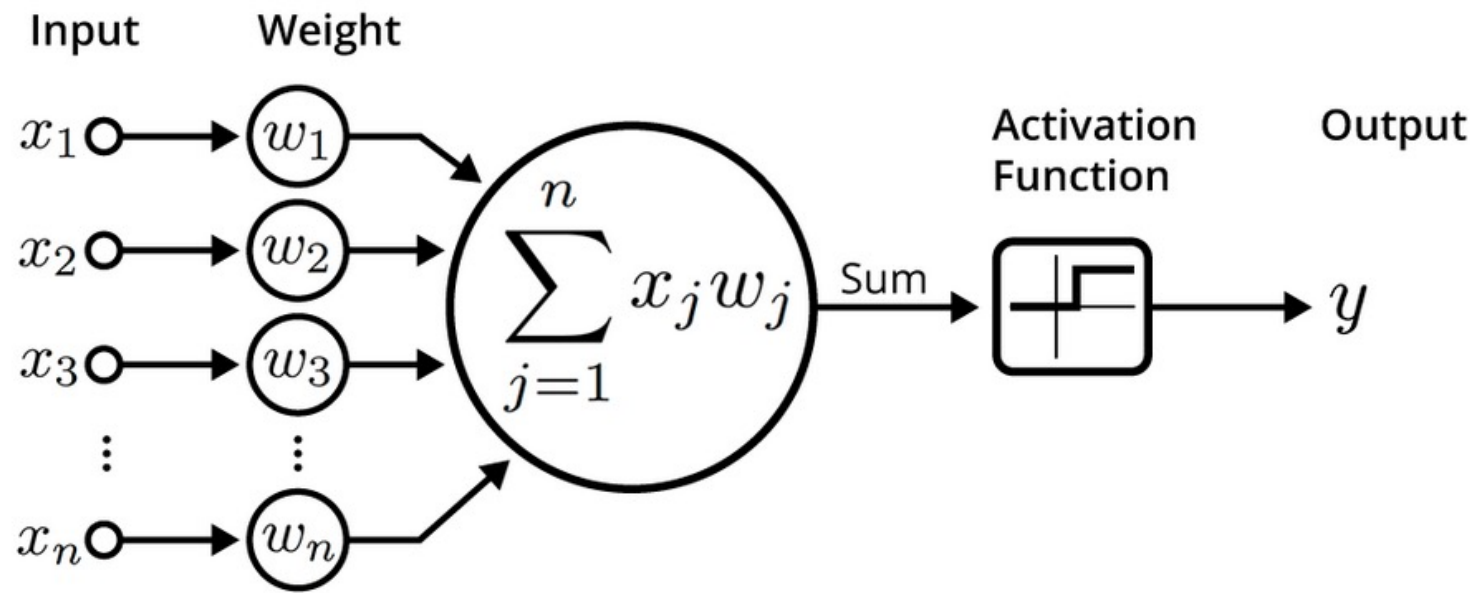


The Math Neuron

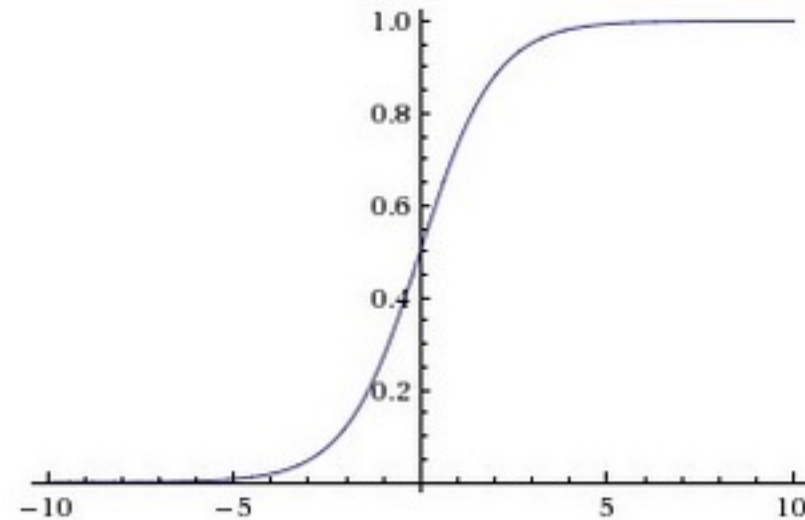


An illustration of an artificial neuron. Source: Becoming Human.

Adding Bias

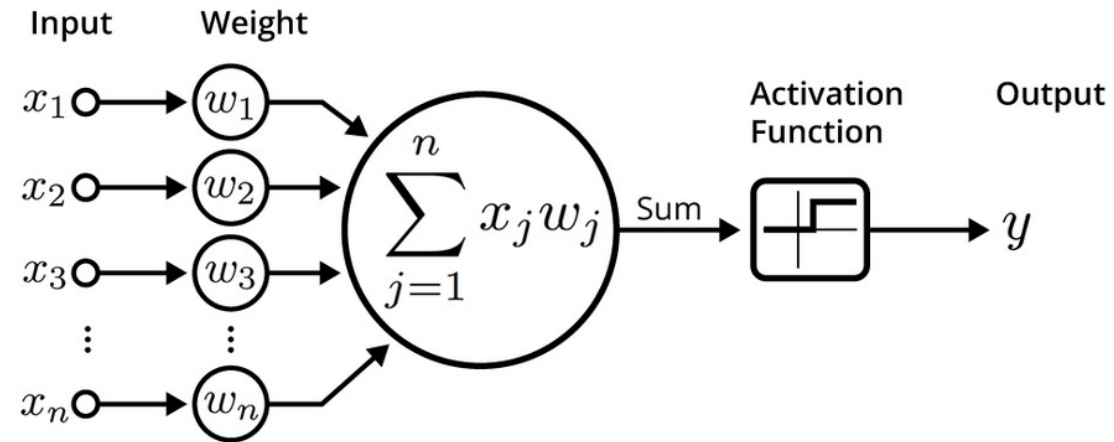


Activation Function: Sigmoid



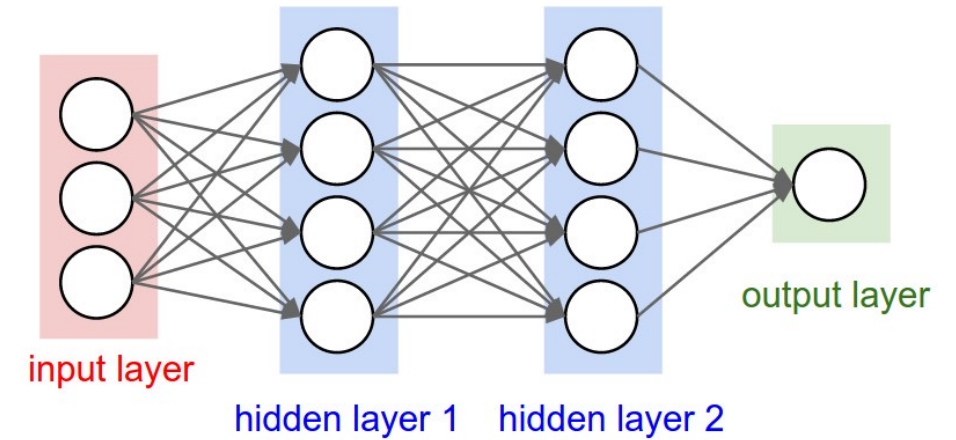
$$\sigma(x) = 1/(1+e^{-x})$$

Python Neuron

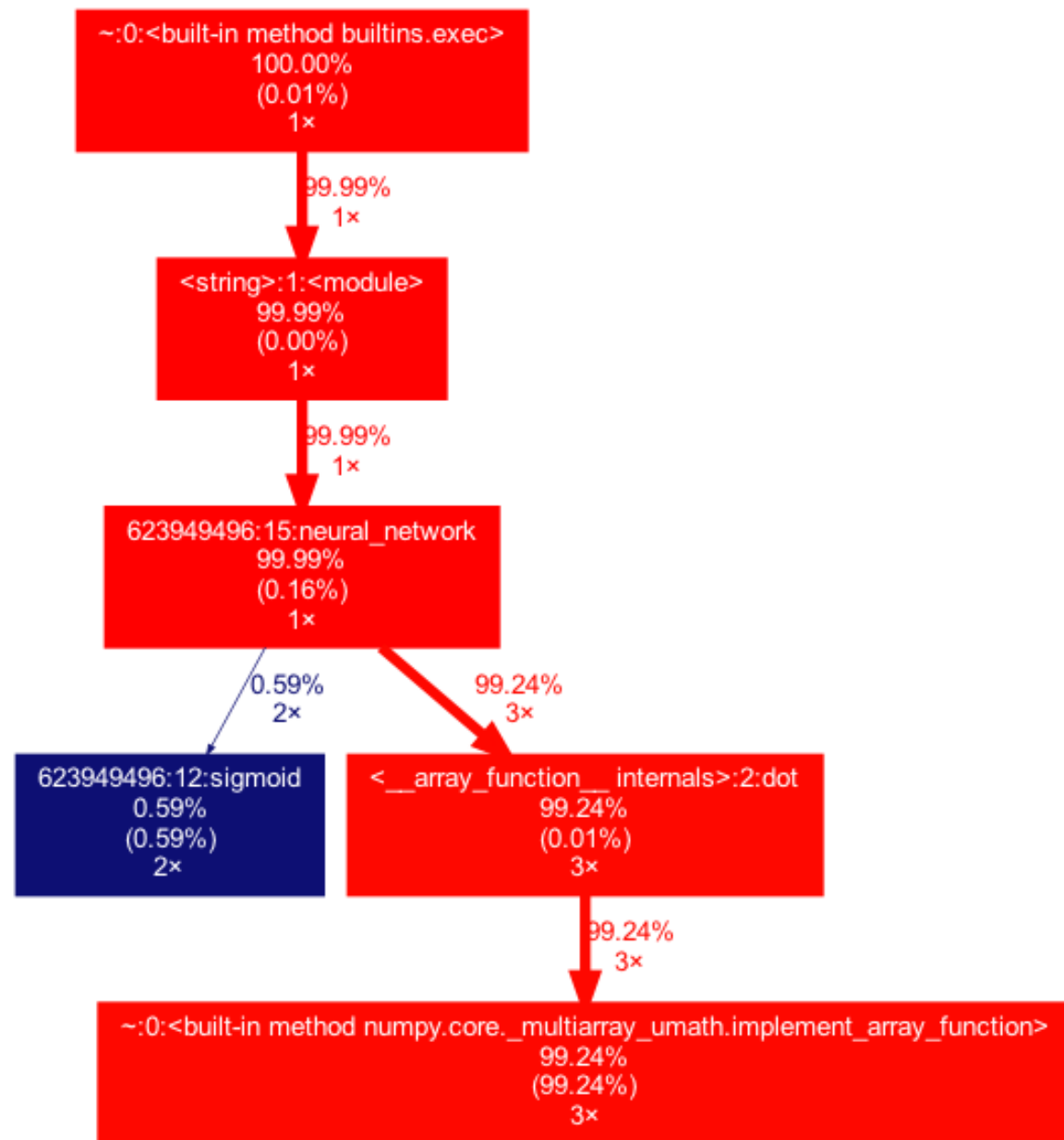


```
class Neuron(object):  
    # ...  
    def forward(self, inputs):  
        """ assume inputs and weights are 1-D numpy arrays and bias is a number """  
        cell_body_sum = np.sum(inputs * self.weights) + self.bias  
        firing_rate = 1.0 / (1.0 + math.exp(-cell_body_sum)) # sigmoid activation function  
        return firing_rate
```

Why focus on Dot Product?



```
# forward-pass of a 3-layer neural network:  
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)  
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)  
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)  
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)  
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```



Matrix Multiplication (Dot Product)

$$\begin{bmatrix} i_0 & i_1 \end{bmatrix} \times \begin{bmatrix} w_{00} & w_{10} & w_{20} \\ w_{01} & w_{11} & w_{21} \end{bmatrix} = \begin{bmatrix} o_0 & o_1 & o_2 \end{bmatrix}$$

$$o_0 = i_0 \cdot w_{00} + i_1 \cdot w_{01}$$

$$o_1 = i_0 \cdot w_{10} + i_1 \cdot w_{11}$$

$$o_2 = i_0 \cdot w_{20} + i_1 \cdot w_{21}$$

Matrix Multiplication (Dot Product)

$$\begin{array}{c} \text{inputs} \\ \underline{[0.1 \quad 0.2]} \end{array} \times \begin{array}{c} \text{weights} \\ \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \end{array} =$$

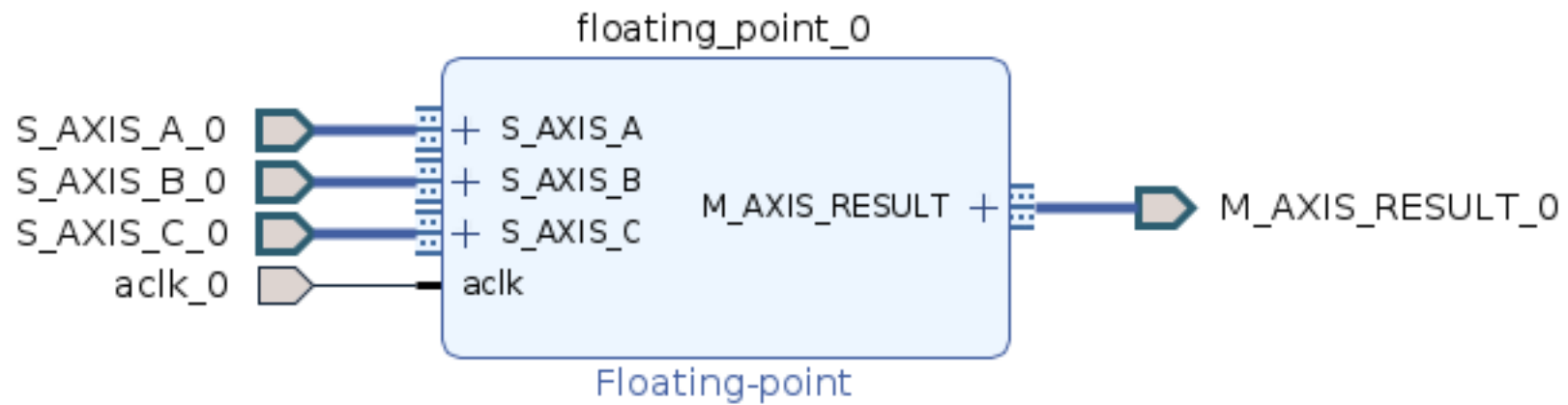
$$= \begin{bmatrix} (0.1 \times 1 + 0.2 \times 4) & (0.1 \times 2 + 0.2 \times 5) & (0.1 \times 3 + 0.2 \times 6) \end{bmatrix}$$

$$= \begin{array}{c} \text{(Answer)} \\ \underline{[0.9 \quad 1.2 \quad 1.5]} \end{array}$$

Floating-Point Multiply-Accumulate (FMAC)

- Math: $a * b + c$

Floating-Point Multiply in Hardware

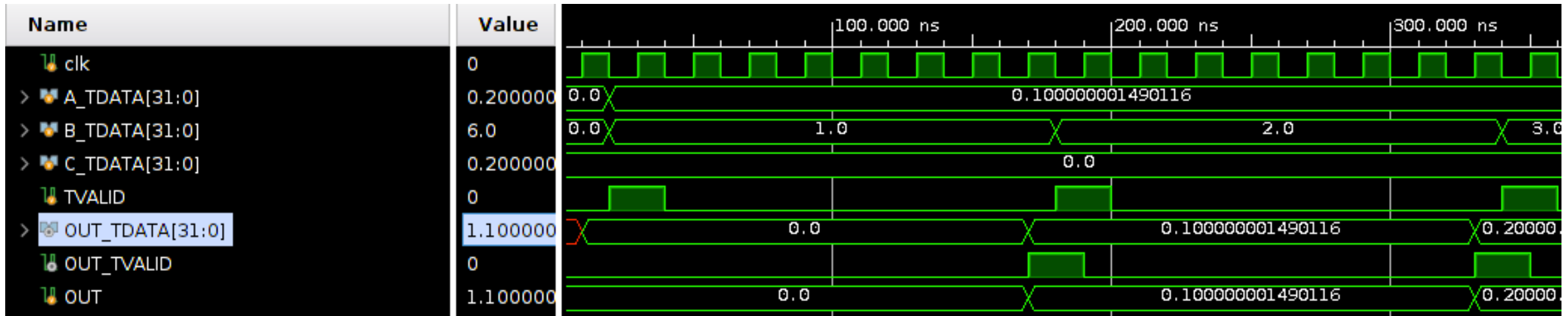


- $\text{result} = a * b + c$

Floating-Point math takes 8 cycles.

- Floating-Point is complicated.
- 8 cycles of complicated.

Demo Time



Floating-Point math takes 8 cycles.

- Floating-Point is complicated.
- 8 cycles of complicated.
- How do we work around an 8 cycle latency?
- Pipelining!

How does Pipelining work in Math?

$$\begin{array}{r} \\ \times 3 \\ \hline 6 \end{array} \Rightarrow \begin{array}{r} \\ \times 11 \\ \hline 110 \end{array} \Rightarrow \begin{array}{r} \\ \times 1 \\ \hline 10 \end{array} + \begin{array}{r} \\ \times 10 \\ \hline 100 \end{array} = 110$$

How does pipelining
work in circuits?

2	10	10	10	
<u>x 3</u>	=> <u>x 11</u>	=> <u>x 1</u>	+	<u>x 10</u>
6	110	10	+	100 = 110

FMAC Pipelining

$$\{ \underline{0.1} \quad 0.2 \} \begin{bmatrix} 1 \underline{2} 3 \\ 4 5 6 \end{bmatrix}$$

	Cycles																			
	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
$0.1 \cdot 1 + 0$																				
$0.1 \cdot 2 + 0$																				
$0.1 \cdot 3 + 0$																				

FMAC Pipelining

Cycles

01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20

Latency vs. Throughput

- **Latency:** How long does an individual operation take to complete?

individual mult: 8 cycles

- **Throughput:** How many operations can you complete per second (or per cycle)?

Average output per cycles:

~ 1 mult / cycle

Pipelining

- FMAC takes 8 cycles for 1 value
 - But can accept a new value every cycle.
-
- What is Latency: 8
 - What is Throughput: $\sim 1/\text{cycle}$

Recall: Matrix Multiplication (Dot Product)

$$\begin{array}{c} \text{inputs} \\ \underline{[0.1 \quad 0.2]} \end{array} \times \begin{array}{c} \text{weights} \\ \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \end{array} =$$

$$= \begin{bmatrix} (0.1 \times 1 + 0.2 \times 4) & (0.1 \times 2 + 0.2 \times 5) & (0.1 \times 3 + 0.2 \times 6) \end{bmatrix}$$

$$= \begin{array}{c} \text{(Answer)} \\ \underline{[0.9 \quad 1.2 \quad 1.5]} \end{array}$$

Alternative Dot Computations

$$\begin{bmatrix} 0.1 & 0.2 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 0.9 & 1.2 & 1.5 \end{bmatrix}$$

$\begin{bmatrix} \underline{0} & \underline{0} & \underline{0} \end{bmatrix}$

$$\left\{ \begin{array}{l} 0.1 \cdot 1 + 0 \\ 0.1 \cdot 2 + 0 \\ 0.1 \cdot 3 + 0 \end{array} \right\}$$

$$\left\{ \begin{array}{l} 0.2 \cdot 4 \\ 0.2 \cdot 5 \\ 0.2 \cdot 6 \end{array} \right\}$$

Multiply-Accumulate Dot Computations

$$\begin{bmatrix} 0.1 & 0.2 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} =$$

Diagram illustrating the first row of the dot product computation. Arrows point from the first row of the matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ to the first row of the vector $\begin{bmatrix} 0.1 & 0.2 \end{bmatrix}$. The first row of the vector is 0.1 and the first row of the matrix is $1 \ 2 \ 3$. The diagram shows the first row of the dot product computation: $0.1 \cdot 1 + 0$, $0.1 \cdot 2 + 0$, and $0.1 \cdot 3 + 0$.

$$0.2 \cdot 4 + 0.1 \quad 0.2 \cdot 5 + 0.2 \quad 0.2 \cdot 6 + 0.3$$

$$\begin{bmatrix} 0.9 & 1.2 & 1.5 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix} = \text{result}$$

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} = \text{result}$$

$$\begin{bmatrix} 0.9 & 1.2 & 1.5 \end{bmatrix}$$

Python Time

$$\begin{array}{c} \text{inputs} \\ [0.1 \quad 0.2 \quad 0.3] \end{array} \times \begin{array}{c} \text{weights} \\ \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} \end{array} =$$

```
weights = np.array( [[1,2,3,4],[5,6,7,8],[9,10,11,12]], dtype=np.float32)
inputs = np.array([[0.1,0.2,0.3]], dtype=np.float32)
outputs = np.dot(inputs, weights)
```

Input		Weights		Output
[0.1 0.2 0.3]	.	[1. 2. 3. 4.]	=	[3.8000002 4.4
		[5. 6. 7. 8.]		5. 5.6000004]
		[9. 10. 11. 12.]		

Input		Weights		Output	
[[0.1 0.2 0.3]]	.	[1. 2. 3. 4.]	=	[3.8000002 4.4	5.
		[5. 6. 7. 8.]			5.6000004]
		[9. 10. 11. 12.]			

Alternative Dot

how its done in dot.sv

```
def pydot(inputs, weights):
    inputs = inputs[0] # remove outer nesting
    outs = np.zeros(weights.shape[1], dtype=np.float32)
    for i in range(weights.shape[0]): # input length
        for j in range(weights.shape[1]): # output length
            outs[j] = outs[j] + weights[i][j] * inputs[i]
    return outs
```

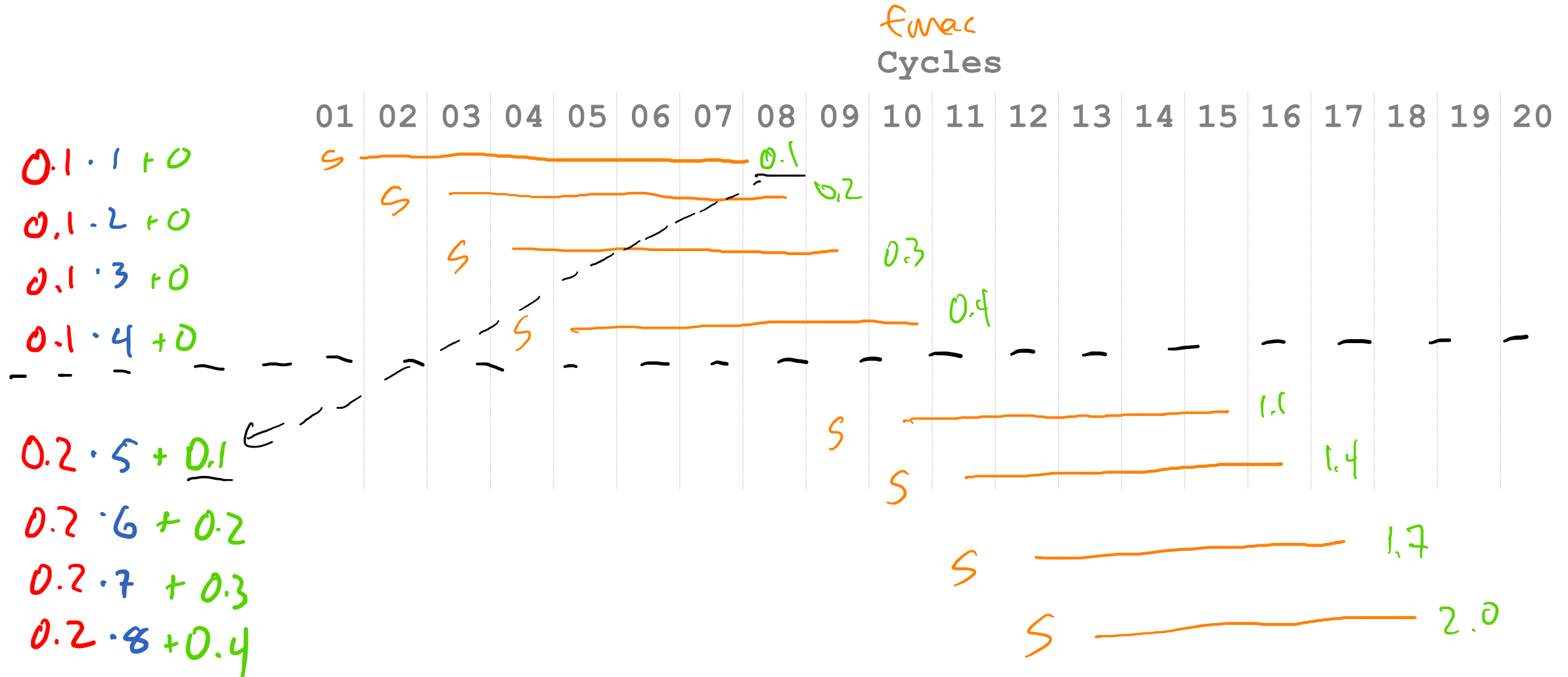
Inputs (Shape):
(1, 3)
Output (Shape):
(1, 4)
Weights (Shape):
(3, 4)

Input
[[0.1 0.2 0.3]]

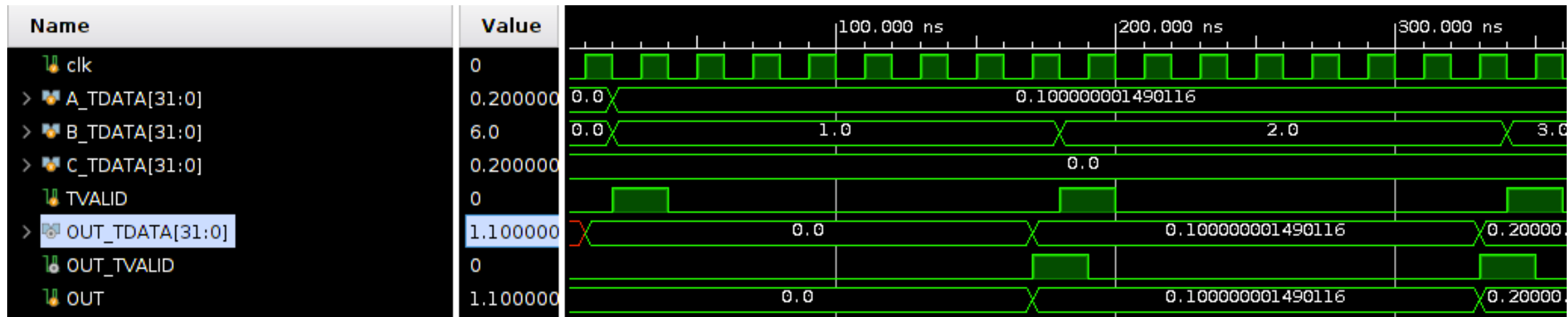
Weights
[1. 2. 3. 4.]
[5. 6. 7. 8.]
[9. 10. 11. 12.]

Output
= [3.8000002 4.4 5. 5.6000004]

Dependencies



Demo Time



Next Time: More on
Dependencies